STAJ PROJESİ · .NET 8 + ANGULAR 21

Todo Mimari Rehberi

Bu rehber, yazdığımız Todo uygulamasını temelden anlatır: Clean Architecture, interface'ler, dependency injection, asenkron yapı, CancellationToken, kimlik doğrulama, yetkilendirme, veri erişimi ve Angular tarafı. Örneklerin hepsi senin projendeki gerçek dosyalardan alındı.

İçindekiler

1. Bir isteğin yolculuğu
2. Clean Architecture
3. Interface'ler
4. Dependency Injection
5. Asenkron programlama
6. CancellationToken
7. Kimlik doğrulama
8. Yetkilendirme ve erişim seviyeleri
9. Validation, hatalar ve loglama
10. Veri erişimi: EF Core
11. Arka plan işleri
12. Frontend temelleri
13. Test stratejisi
14. Kendini test et
15. Sözlük

Her konu aynı sırayla anlatılıyor: önce kavram, sonra projede nerede olduğu, sonra neden öyle yapıldığı, en son da tuzaklar. Katmanlar sayfa boyunca hep aynı renkle gösteriliyor:

DOMAİN APPLİCATİON INFRASTRUCTURE API WEB (ANGULAR) SQL SERVER

Hazırlanma tarihi: 6 Ekim 2026 · Etkileşimli sürüm: claude.ai/artifact/Rrx39fSfxr6XR2r1D2QQDw

01 Bir isteğin yolculuğu

Mimariyi tek tek parçalarıyla anlatmadan önce bütüne bakalım. Kullanıcı "Görevi oluştur" butonuna bastığında kod hangi dosyalardan geçiyor? Aşağıdaki 12 adım isteği baştan sona izliyor. Her adımın başlığındaki renkli etiket, o adımın hangi katmanda olduğunu gösteriyor.

WEB Adım 1 / 12 Component butonu yakalar

Form gönderilince `TodoPage.createTodo()` çalışır. Önce kendi basit kontrolünü yapar (başlık boş mu), sonra isteği bir servise devreder. Component HTTP'nin nasıl yapıldığını bilmez, sadece "bunu oluştur" der.

WEB todo.Web/src/app/pages/todos/todo-page.ts

```
const request = this.createTodoRequest(title, this.newTodoDescription, this.newTodoDueAtLocal,
this.newTodoPriority);
const createdTodo = await firstValueFrom(
  this.todoApi.createTodo(todoListId, request),
);
this.todos = [...this.todos, createdTodo];
```

WEB Adım 2 / 12 Servis HTTP isteğini hazırlar

`TodoApiService` sadece adres ve yöntem bilgisini taşır. `HttpClient.post` bir **Observable** döndürür. İstek henüz gitmez. Biri abone olduğunda gider; burada abone olan `firstValueFrom`.

WEB todo.Web/src/app/services/todo-api.service.ts

```
createTodo(todoListId: number, request: TodoRequest): Observable<TodoItem> {
  return this.http.post<TodoItem>(`${this.apiUrl}/todo-lists/${todoListId}/todos`, request);
}
```

WEB Adım 3 / 12

Interceptor token ekler

Her istek yola çıkmadan `authInterceptor` içinden geçer. İstek bizim API'mize gidiyorsa `Authorization: Bearer <token>` başlığını ekler. Token'ın süresi dolmuşsa önce sessizce yeniler. Servislerin hiçbiri token'la uğraşmaz.

WEB todo.Web/src/app/interceptors/auth.interceptor.ts

```
const accessToken = auth.getAccessToken();
const firstAttempt =
  accessToken !== null
    ? sendWithToken(preparedRequest, next, accessToken)
    : auth.refreshSession().pipe(switchMap((session) => sendWithToken(preparedRequest, next,
session?.accessToken ?? null)));
```

API Adım 4 / 12

Tarayıcı ve CORS

Sayfa `localhost:4200`'de, API `localhost:7070`'de. Port farklı olduğu için tarayıcı bunu **farklı köken (origin)** sayar. İstekte `Authorization` başlığı olduğundan tarayıcı önce bir `OPTIONS` isteği (preflight) gönderip "bu siteden istek kabul ediyor musun?" diye sorar. API'deki CORS ayarı sadece listedeki adreslere "evet" der.

API todo.API/Program.cs

```
policy
    .WithOrigins(allowedOrigins)
    .WithHeaders("Content-Type", "Authorization", RequireClientHeaderAttribute.HeaderName)
    .WithMethods("GET", "POST", "PUT", "PATCH", "DELETE")
    .AllowCredentials();
```

API Adım 5 / 12

Middleware hattı

İstek controller'a varmadan, `Program.cs`'te eklendikleri sırayla bir dizi **middleware**'den geçer. Her biri isteğe bir şey yapar ve sonrakine verir, dönüşte de yanıta bir şey ekleyebilir. Bir sonraki bölümde bu hattı ayrıntılı göreceğiz.

API todo.API/Program.cs (sıra)

```
app.UseSecurityHeaders();          // güvenlik başlıkları
app.UseSerilogRequestLogging();    // her isteği tek satır logla
app.UseExceptionHandler();         // hataları ProblemDetails'e çevir
app.UseCors("TodoWeb");            // köken kontrolü
app.UseRateLimiter();              // giriş denemelerini sınırla
app.UseAuthentication();           // token'ı doğrula: kimsin?
app.UseAuthorization();            // politikaları uygula: iznin var mı?
app.MapControllers();              // doğru controller'ı bul
```

API Adım 6 / 12

Controller: HTTP'den command'a

ASP.NET adres ve JSON gövdeyi parametrelere doldurur (**model binding**). Controller'ın tek işi HTTP dünyasını (route, body, durum kodu) uygulama dünyasına (command, use case) çevirmektir. İçinde iş kuralı yoktur, bu yüzden "ince controller" denir.

API `todo.API/Controllers/TodosController.cs`

```
[HttpPost]
public async Task<ActionResult<TodoItemDto>> Create(
    [FromRoute] int todoListId,
    [FromBody] CreateTodoRequest request,
    CancellationToken cancellationToken)
{
    CreateTodoCommand command = new CreateTodoCommand
    {
        TodoListId = todoListId,
        Title = request.Title,
        Description = request.Description,
        DueAtUtc = request.DueAtUtc,
        Priority = request.Priority
    };

    TodoItemDto? createdTodo = await _createTodoUseCase.ExecuteAsync(command, cancellationToken);

    if (createdTodo == null)
    {
        return NotFound();
    }

    return CreatedAtAction(nameof(GetById), new { todoListId, todoId = createdTodo.Id },
createdTodo);
}
```

APPLİCATİON Adım 7 / 12

Use case: önce validation

Use case'in ilk satırı veriyi doğrular. Başlık boşsa veya çok uzunsa iş mantığına hiç girilmez. FluentValidation bir `RequestValidationException` fırlatır, API bunu 400'e çevirir.

APPLİCATİON `todo.Application/Features/Todos/CreateTodo/CreateTodoUseCase.cs`

```
await _validator.ValidateOrThrowAsync(command, cancellationToken);
```

INFRASTRUCTURE Adım 8 / 12

Repository listeyi yükler

Use case `ITodoListRepository` interface'ini çağırır, gerçekte çalışan ise Infrastructure'daki EF Core sınıfıdır. `Include`, listenin görevlerini de aynı sorguda getirir. "Aynı isimli görev var mı?" kontrolü için bu gerekli.

INFRASTRUCTURE `todo.Infrastructure/Persistence/TodoListRepository.cs`

```
IQueryable<TodoList> todoListQuery = _todoDbContext.TodoLists;
todoListQuery = todoListQuery.Include(todoList => todoList.Todos);

TodoList? todoList = await todoListQuery.FirstOrDefaultAsync(
    todoList => todoList.Id == todoListId,
    cancellationToken);
```

APPLİCATİON Adım 9 / 12

Yetki kontrolü

Liste bulundu, ama bu kullanıcının mı? `TodoListAccessPolicy` listenin sahibine ya da Admin'e izin verir. Başkasının listesiyse use case `null` döner ve controller 404 verir.

APPLİCATİON `todo.Application/Features/Todos/CreateTodo/CreateTodoUseCase.cs`

```
bool canAccessTodoList = TodoListAccessPolicy.CanAccess(todoList, _currentUser);

if (canAccessTodoList == false)
{
    return null;
}
```

DOMAİN Adım 10 / 12

Domain kuralı uygular

Görevi listenin kendisi ekler. `TodoItem`'ın constructor'ı başlığı doğrular ve kırpar. `TodoList` aynı isimli başka bir görev olup olmadığına bakar. Kural ihlal edilirse `DuplicateNameException` fırlatılır.

DOMAİN `todo.Domain/Entities/TodoList.cs`

```
TodoItem todo = new TodoItem(Id, title, description, dueAtUtc, priority);

EnsureTitleIsUnique(todo.Title, todoBeingRenamed: null);

_todos.Add(todo);
```

SQL SERVER Adım 11 / 12

Kayıt: SaveChanges ve SQL

Use case `SaveChangesAsync` çağırır. EF Core yeni eklenen `TodoItem`'ı fark eder ve bir `INSERT` üretir. Veritabanındaki unique index son kontrolü yapar. SQL Server'dan bir hata gelirse `TodoDbContext` onu teknolojiden bağımsız bir exception'a çevirir.

INFRASTRUCTURE todo.Infrastructure/Persistence/TodoDbContext.cs

```
try
{
    return await base.SaveChangesAsync(acceptAllChangesOnSuccess, cancellationToken);
}
catch (DbUpdateException exception)
{
    throw DatabaseErrorTranslator.Translate(exception);
}
```

WEB Adım 12 / 12

Geri dönüş: DTO, 201, ekran

Use case entity'yi değil bir **DTO** (`TodoItemDto`) döndürür. Controller bunu `201 Created` ile JSON'a çevirir. Angular'da `firstValueFrom` değeri verir, component listeyi günceller ve `markForCheck()` ile ekranı yeniletir. Herhangi bir adımda hata olursa yanıt, 9. bölümdeki hata handler'larından birinden döner.

WEB HTTP yanıtı

```
HTTP/1.1 201 Created
Location: /api/todo-lists/3/todos/12

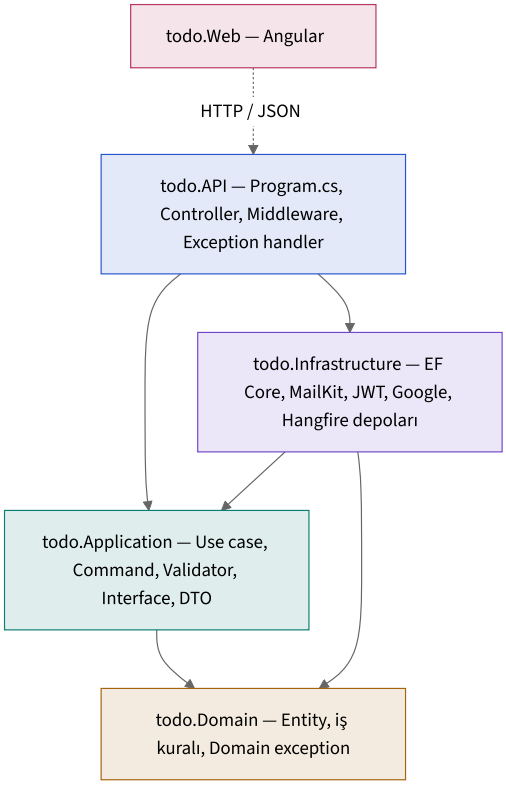
{ "id": 12, "todoListId": 3, "title": "Raporu yaz", "description": null,
  "isCompleted": false, "dueAtUtc": null, "priority": "Normal", "createdAtUtc": "2026-10-
06T15:02:11Z" }
```

BU BÖLÜMDEN ÇIKARILACAK

Her dosyanın tek bir işi var. Component ekranla, servis HTTP ile, controller HTTP ile uygulama arasındaki çeviriyle, use case iş akışıyla, entity kurallarla, repository veritabanıyla ilgileniyor. Bir şeyi değiştirmen gerektiğinde nereye bakacağını bilmenin anahtarı bu ayrım. Sonraki bölümlerin hepsi bu ayrımın neden ve nasıl yapıldığını anlatıyor.

02 Clean Architecture

Clean Architecture, kodu **sorumluluklarına göre katmanlara** ayıran ve bu katmanlar arasındaki bağımlılıklara tek bir kural koyan bir mimari yaklaşım. O kurala **bağımlılık kuralı** deniyor: *kod yalnızca içe doğru bağımlı olabilir.* En içte iş kuralları durur, en dışta veritabanı, web, e-posta gibi teknik detaylar.



Oklar "kim kimi tanıyor" yönünü gösterir. Domain hiçbir projeye referans vermez. Application sadece Domain'i bilir. Bunu `.csproj` dosyalarındaki `ProjectReference` satırlarında görebilirsin.

Katmanlar tek tek

KATMAN	NE İÇERİR	NEYİ BİLMEZ	PROJEDEKİ ÖRNEKLER
DOMAİN	İşin kendisi: varlıklar (entity), değişmez kurallar, domain hataları	Veritabanını, HTTP'yi, JSON'u, hatta Application'ı	`TodoList`, `TodoItem`, `User`, `RefreshToken`, `DuplicateNameException`
APPLİCATİON	Uygulamanın yaptığı işler (use case), giriş verisi (command), doğrulama, dış dünyadan beklentiler (interface), çıkış verisi (DTO)	SQL Server'ı, EF Core'u, ASP.NET'i, MailKit'i	`CreateTodoUseCase`, `CreateTodoCommandValidator`, `ITodoListRepository`, `IEmailSender`, `TodoItemDto`
INFRASTRUCTURE	Application'daki interface'lerin gerçek, teknolojiye bağlı halleri	HTTP isteklerini, controller'ları	`TodoListRepository` (EF Core), `MailKitEmailSender`, `JwtAccessTokenGenerator`, `GoogleIdTokenValidator`
API	Uygulamanın giriş kapısı ve her şeyi birbirine bağlayan yer (composition root)	İş kurallarının detayını (sadece use case'i çağırır)	`Program.cs`, controller'lar, exception handler'lar, `HttpContextCurrentUser`, Hangfire kurulumu

Domain: kuralları koruyan zengin model

Domain sınıflarının en önemli özelliği **geçersiz duruma düşürülememeleri**. Bunun için iki teknik birlikte kullanılıyor:

- **Private setter'lar:** `todo.Title = ""` yazmak derlenmez. Değer ancak `Rename` gibi kuralı kontrol eden bir metotla değişebilir.
- **Constructor'da doğrulama:** `new TodoList("", 1)` anında `DomainRuleViolationException` fırlatır. Geçersiz bir nesne hiç oluşmaz.

DOMAİN todo.Domain/Entities/TodoList.cs (kısaltılmış)

```
public class TodoList
{
    private readonly List<TodoItem> _todos = new List<TodoItem>();

    public string Name { get { return _name; } private set { _name = value; } }

    // Dışarıya yalnızca okunabilir bir görünüm verilir; listeye dışarıdan eleman eklenemez.
    public IReadOnlyCollection<TodoItem> Todos { get { return _todos.AsReadOnly(); } }

    public TodoItem AddTodo(string title, ...)
    {
        TodoItem todo = new TodoItem(Id, title, description, dueAtUtc, priority);
        EnsureTitleIsUnique(todo.Title, todoBeingRenamed: null);
        _todos.Add(todo);
        return todo;
    }
}
```

Buna **zengin domain modeli** deniyor. Bunun tersi, sadece `{ get; set; }` 'lerden oluşan ve bütün kuralların servislerde dağınık durduğu **anemik model**. Anemik modelde aynı kuralı unutmak çok kolaydır. Zengin modelde kural nesnenin içinde olduğu için atlanamaz.

Aggregate root: TodoList

`TodoList` ile görevleri birlikte bir **aggregate** oluşturur. Bu grubun dışarıya açılan kapısı, yani **aggregate root**, `TodoList` 'tir. Görev eklemek, silmek ve yeniden adlandırmak liste üzerinden yapılır. "Aynı listede aynı isimli görev olamaz" kuralı bu sayede uygulanabiliyor, çünkü tek bir görev kardeşlerini bilmez ama liste hepsini bilir. Bu yüzden `TodoItem.Rename` metodu `internal` yapıldı: Domain projesinin dışından çağrılamaz, tek yol `TodoList.RenameTodo` .

Application: use case başına bir sınıf

Application katmanı `Features` klasöründe **işe göre** düzenlenmiş. Bu yapıya "vertical slice" (dikey dilim) da denir. Her iş kendi klasöründe üç dosyayla durur:

- `CreateTodoCommand.cs` : use case'e giren veri. API'deki `CreateTodoRequest` ile aynı görünür ama bilerek ayrı tutulur. Use case yarın HTTP'den değil bir arka plan işinden çağrılırsa command aynı kalır.
- `CreateTodoCommandValidator.cs` : verinin biçim kuralları.
- `CreateTodoUseCase.cs` : işin adımları. Doğrula, yükle, yetkiyi kontrol et, domain'e uygula, kaydet, DTO döndür.

Use case asla entity döndürmez, **DTO** döndürür. Üç sebebi var: entity'nin her alanı dışarı açılmamalı (`User.PasswordHash` gibi), JSON'a çevrilirken EF Core'un tembel yükleme ve döngüsel referans sorunları çıkmamalı, ve API'nin sözleşmesi iç model değişince bozulmamalı.

Infrastructure ve API

Infrastructure, Application'ın "bana şu lazım" dediği interface'leri gerçek teknolojiyle doldurur. API ise **composition root**: bütün parçaların birbirine bağlandığı tek yer. `Program.cs` şunları çağırır:

API todo.API/Program.cs

```
builder.Services.AddApplication();                                  // use case'ler, validator'lar
builder.Services.AddInfrastructure(builder.Configuration);          // EF Core, repository'ler, mail, JWT
builder.Services.AddTodoAuthentication(builder.Configuration);      // JWT doğrulama, politikalar
builder.Services.AddTodoBackgroundJobs(builder.Configuration);      // Hangfire
```

Bağımlılığı tersine çevirmek (Dependency Inversion)

Burada çoğu kişinin kafası karışıyor: use case veritabanından veri okuyor, yani veritabanına bağımlı olmalı değil mi? Clean Architecture'ın püf noktası bu bağımlılığı **tersine çevirmek**.

1. Application kendi ihtiyacını bir interface olarak yazar: `ITodoListRepository`. Bu interface Application projesinde durur.
2. Infrastructure bu interface'i implement eder: `TodoListRepository : ITodoListRepository`. Infrastructure, Application'a bağımlıdır.
3. Çalışma anında (runtime) DI container use case'e gerçek repository'yi verir.

Sonuçta **çalışma akışı** dışarı doğru (use case → repository → SQL), **kod bağımlılığı** içeri doğru (Infrastructure → Application). Use case EF Core'u hiç tanımıyor. Yarın veritabanı PostgreSQL olsa, ya da testlerde bellekteki sahte bir liste kullanılsa, use case'in tek satırı değişmez.

BENZETME

Duvardaki priz bir interface'tir. Ütü (use case) prize uyan bir fiş ister. Prizin arkasında barajdan mı, güneş panelinden mi elektrik geldiğini bilmez. Elektrik şirketi (Infrastructure) prizin standardına uymak zorundadır, ütü elektrik şirketine uymak zorunda değildir.

SIK YAPILAN HATALAR

- **Domain'e `[Required]`, `[Table]` gibi attribute'lar koymak.** Bunlar ASP.NET ve EF Core'a ait. Bizim projede EF eşlemesi `TodoDbContext.OnModelCreating`'de duruyor, Domain temiz kalıyor.
- **Controller'a iş kuralı yazmak.** "Liste sahibi mi?" kontrolü controller'da olsaydı, aynı use case'i Hangfire'dan çağırdığında kontrol atlanırdı.
- **Use case'te `DbContext` kullanmak.** Application, Infrastructure'ı tanımamalı. Tanırsa test etmek için gerçek bir veritabanı gerekir.
- **Her sınıfa interface açmak.** Interface'i yalnızca değiştirilebilmesi gereken veya katman sınırını aşan bağımlılıklara aç (bir sonraki bölüm).

PATRON SORARSA: "NEDEN BU KADAR KATMAN?"

"İş kuralları teknolojiden bağımsız olsun diye. Domain ve Application testleri veritabanı olmadan milisaniyede çalışıyor; şu an 136 tanesi öyle. Veritabanı, mail servisi veya kimlik sağlayıcı değişse iş kuralları değişmiyor."

03 Interface'ler

Interface bir **sözleşmedir**: "Bu işi yapan biri şu metotlara sahip olmalı" der ama işin nasıl yapılacağını söylemez. Kodu yazan kişi interface'e güvenir, arkasında hangi sınıfın çalıştığını bilmez.

APPLİCATİON todo.Application/Abstractions/Email/IEmailSender.cs

```
public interface IEmailSender
{
    bool IsConfigured { get; }

    Task SendAsync(EmailMessage message, CancellationToken cancellationToken);
}
```

Hatırlatma use case'i sadece `IEmailSender`'ı tanır. Uygulamada arkasında MailKit ile Gmail çalışır, testte ise gönderilen mailleri bir listeye yazan `FakeEmailSender`. Kod aynı, davranış değişiyor.

Projedeki interface'ler ve neden var oldukları

INTERFACE	TANIMLANDIĞI YER	GERÇEK İMPLEMENTASYON	NEDEN İNTERFACE?
`ITodoListRepository`, `IUserRepository`, `IRefreshTokenRepository`, `ITodoReminderRepository`	APP	INFRA EF Core repository'leri	Use case veritabanı teknolojisini bilmesin. Testlerde bellekteki listeyle değiştirilsin.
`ICurrentUser`	APP	API `HttpContextCurrentUser`	"İsteği kim yaptı?" bilgisi HTTP'den, yani token'dan gelir. Application HTTP'yi bilmez.
`IPasswordHasher`	APP	INFRA ASP.NET Identity hasher'ı	Kriptografi detayı dışarıda kalsın. Testte okunabilir sahte hash kullanılsın.
`IAccessTokenGenerator`, `ISecureTokenGenerator`	APP	INFRA JWT ve kriptografik rastgelelik	Token formatı değişebilir. Testte tahmin edilebilir token gerekir.
`IGoogleTokenValidator`	APP	INFRA Google.Apis.Auth	Testte gerçek Google'a gitmeden "geçerli token" senaryosu kurulabilsin.
`IRoleAssignmentPolicy`	APP	INFRA yapılandırmadan okuyan sınıf	Admin listesi appsettings veya User Secrets'tan gelir. Application yapılandırmayı bilmez.
`IEmailSender`	APP	INFRA `MailKitEmailSender`	Yarın SendGrid'e geçilirse sadece bir sınıf değişir.
`IValidator<T>`	FluentValidation	Bizim validator sınıflarımız	Kütüphanenin kendi interface'i. DI container'dan doğru validator'ı ister.
`IExceptionHandler`	ASP.NET Core	API beş handler	Framework'ün sözleşmesi. Hatayı tanıyan handler `true` döner.

Interface'in en büyük kazancı: test

Uygulama testlerinde gerçek veritabanı yok. Use case'e interface'in sahte bir implementasyonu veriliyor:

APPLİCATİON.TESTS todo.Tests/todo.Application.Tests/DuplicateNameUseCaseTests.cs

```
FakeTodoListRepository repository = new FakeTodoListRepository();   // ITodoListRepository'nin
bellekteki hali
repository.AddExistingTodoList("Ev işleri");
CreateTodoListUseCase useCase = new CreateTodoListUseCase(
    repository, FakeCurrentUser.ListOwner(), new CreateTodoListCommandValidator());

await Assert.ThrowsAsync<DuplicateNameException>(
    () ⇒ useCase.ExecuteAsync(new CreateTodoListCommand { Name = "  EV İŞLERİ " }, ct));
```

Bu test milisaniyeler içinde çalışır, SQL Server kapalıyken bile. Bizim projede bunlara **fake** deniyor: interface'i basit ama gerçekten çalışan bir şekilde implement eden sınıflar. Moq gibi kütüphanelerle anlık oluşturulan sahte nesnelere de **mock** denir. İkisi de "test double" ailesindendir.

Her şeye interface gerekmez

Projede bilerek interface'i olmayan sınıflar da var:

- `TodoListAccessPolicy` bir `static` sınıf. Sadece saf bir kural ("sahibi veya admin"), dışa bağımlılığı yok. Değiştirilmesi gerekmeyen bir kuralı interface arkasına saklamak sadece karmaşıklık ekler.
- `SessionIssuer` ve use case'lerin kendisi somut sınıf. Controller'lar onları doğrudan ister. Use case'in başka bir implementasyonu olması beklenmiyor.

NE ZAMAN İNTERFACE AÇMALI?

Üç sorudan biri "evet" ise: Bu bağımlılık bir katman sınırını aşıyor mu (Application → Infrastructure)? Dış dünyaya (ağ, disk, saat, rastgelelik) mı dokunuyor? Testte farklı davranması gerekiyor mu? Saat bile buna dahil: proje `DateTime.UtcNow` yerine .NET 8'in `TimeProvider` soyutlamasını kullanıyor. Böylece testlerde "8 gün sonrasına git" diyerek refresh token'ın süresinin dolmasını sınayabiliyoruz.

04 Dependency Injection

Interface'ler "neye ihtiyacım var" sorusunu cevaplıyor. Dependency Injection (DI) ise "bu ihtiyacı kim, ne zaman karşılayacak" sorusunu cevaplıyor.

Problem: new her yerde

DI olmasaydı use case kendi bağımlılıklarını kendisi oluştururdu:

KÖTÜ ÖRNEK böyle yazmıyoruz

```
public class CreateTodoUseCase
{
    private readonly TodoListRepository _repository =
        new TodoListRepository(new TodoDbContext(new DbContextOptionsBuilder<TodoDbContext>()
            .UseSqlServer("Server=localhost\\SQLEXPRESS;...").Options));
}
```

Bu kodda use case hem SQL Server'ı hem bağlantı cümlesini biliyor, test edilemiyor ve aynı `DbContext` diğer sınıflarla paylaşılamıyor. Bağımlılık kuralı da çiğneniyor.

Çözüm: constructor injection

Sınıf ihtiyaçlarını constructor parametresi olarak **ilan eder**, nasıl oluşturulduklarıyla ilgilenmez:

APPLİCATİON todo.Application/Features/Todos/CreateTodo/CreateTodoUseCase.cs

```
public CreateTodoUseCase(
    ITodoListRepository todoListRepository,
    ICurrentUser currentUser,
    IValidator<CreateTodoCommand> validator)
{
    if (todoListRepository == null)
    {
        throw new ArgumentNullException(nameof(todoListRepository));
    }
    // ...
    _todoListRepository = todoListRepository;
}
```

Bu nesneleri oluşturup birbirine veren yapıya **DI container** denir. ASP.NET Core'un kendi container'ı var. Uygulama açılırken ona "şu interface istenirse şu sınıfı ver" diye kayıt yapılır:

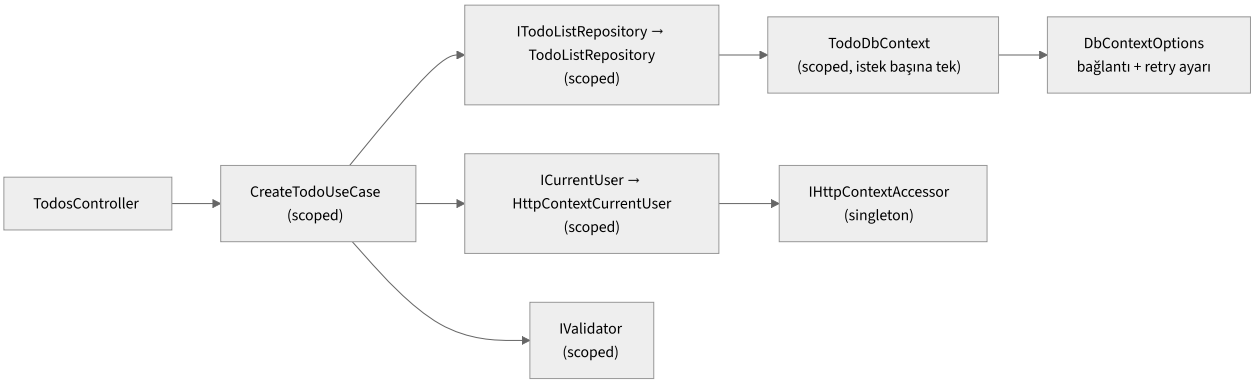
INFRASTRUCTURE todo.Infrastructure/DependencyInjection.cs

```
services.AddDbContext<TodoDbContext>(options ⇒ options.UseTodoSqlServer(connectionString));
services.AddScoped<ITodoListRepository, TodoListRepository>();
services.AddSingleton<IPasswordHasher, AspNetCorePasswordHasher>();
services.AddSingleton<IEmailSender, MailKitEmailSender>();
```

Constructor'lardaki `null` kontrolleri fazlalık gibi görünebilir, çünkü DI zaten `null` vermez. Ama sınıf testlerde elle de oluşturuluyor. Kontrol, yanlış kullanıldığında hatanın sınıfın ilk satırında ve anlaşılır bir mesajla çıkmasını sağlıyor.

Container nesne ağacını nasıl kurar?

Bir HTTP isteği `TodosController`'a geldiğinde container şu ağacı kendisi çözer:



Aynı istekte `TodoDbContext`'i isteyen herkes aynı örneği alır. Bu yüzden bir use case'te iki repository kullansan bile tek bir `SaveChangesAsync` hepsini kaydeder.

Yaşam süreleri (lifetime)

Container'a sadece "ne verilecek" değil, "ne kadar yaşayacak" da söylenir. Bu, DI'ın en çok hata yapılan kısmı.

LİFETİME	ANLAMI	PROJEDE KULLANANLAR	NEDEN
Singleton	Uygulama boyunca tek örnek	Bütün `*Settings` sınıfları, `IPasswordHasher`, `IAccessTokenGenerator`, `IEmailSender`, `IGoogleTokenValidator`, `TimeProvider`, exception handler'lar	Durum (state) tutmuyorlar veya sadece ayar okuyorlar. Her istekte yeniden oluşturmanın anlamı yok.
Scoped	Her HTTP isteği (veya Hangfire işi) için bir örnek	`TodoDbContext`, bütün repository'ler, use case'ler, validator'lar, `ICurrentUser`, `SessionIssuer`, `TodoReminderJob`	`DbContext` o isteğin değişikliklerini takip eder ve thread-safe değildir. `ICurrentUser` o isteğin kullanıcısıdır.
Transient	Her istendiğinde yeni örnek	Projede bilerek kullanılmadı	Hafif ve durumsuz sınıflar için uygundur.

TUZAK: CAPTİVE DEPENDENCY (TUTSAK BAĞIMLILIK)

Bir singleton, scoped bir şeye bağımlı olamaz. Olursa scoped nesne singleton'ın içinde "tutsak" kalır: ilk isteğin `DbContext`'i uygulama kapanana kadar yaşar, bütün kullanıcılar aynı nesneyi paylaşır ve hatalar başlar. ASP.NET Development ortamında bunu fark edip uygulamayı başlatmaz.

Projede iki somut örneği var:

- **Exception handler'lar:** `AddExceptionHandler` onları singleton olarak kaydeder. O yüzden sadece `ILogger` gibi singleton şeylere bağımlılar, repository'ye bağımlı değiller.
- **Arka plan işleri:** Hangfire her iş için yeni bir scope açar. Bu sayede `TodoReminderJob` kendi taze `DbContext`'ini alır. İlk denemedeki `BackgroundService` da singleton olduğu için her turda `IServiceScopeFactory.CreateScope()` ile elle scope açıyordu.

Her katman kendi kaydını yapar

`AddApplication()`, `AddInfrastructure()` gibi extension metotlar sayesinde `Program.cs` hangi repository'nin hangi sınıfla eşleştiğini bilmek zorunda kalmaz. Infrastructure'ın iç detayları Infrastructure'da kalır. FluentValidation'ın `AddValidatorsFromAssemblyContaining` metodu da projedeki bütün validator sınıflarını kendisi bulup kaydeder.

Derinlemesine: ayarlar neden IOptions değil de sınıf olarak kaydedildi?

.NET'te ayarlar genelde `IOptions<T>` ile okunur. Bu projede `JwtSettings.FromConfiguration(...)` gibi metotlarla ayarlar uygulama açılırken bir kez okunuyor, doğrulanıyor ve singleton olarak kaydediliyor. Bunun iki faydası var: eksik bir ayar (örneğin JWT anahtarı) uygulama **açılırken** anlaşılır bir hatayla durur (fail fast), ve sınıflar `IOptions` yerine doğrudan `JwtSettings` ister, daha okunaklı olur. Dezavantajı, ayar dosyası değişince uygulamanın yeniden başlatılması gerekmesi. `IOptionsMonitor` bunu canlı izleyebilir.

Angular'da da aynı fikir

Angular'ın da kendi DI sistemi var. `@Injectable({ providedIn: 'root' })` bir servisi uygulama boyunca tek örnek (singleton) yapar. Component'ler ihtiyaçlarını `inject()` fonksiyonuyla ister:

WEB todo.Web/src/app/pages/todos/todo-page.ts

```
export class TodoPage implements OnInit {
  private readonly todoApi = inject(TodoApiService);
  private readonly messages = inject(NzMessageService);
  protected readonly auth = inject(AuthService);
```

Testlerde de aynı mantık: `TestBed`'e "`TodoApiService` istenirse şu sahte nesneyi ver" denir (`{ provide: TodoApiService, useValue: {...} }`).

05 Asenkron programlama

Projedeki neredeyse her metot `async` ve `Task` döndürüyor. Nedenini anlamak için önce bir web sunucusunun zamanını nereye harcadığına bakalım.

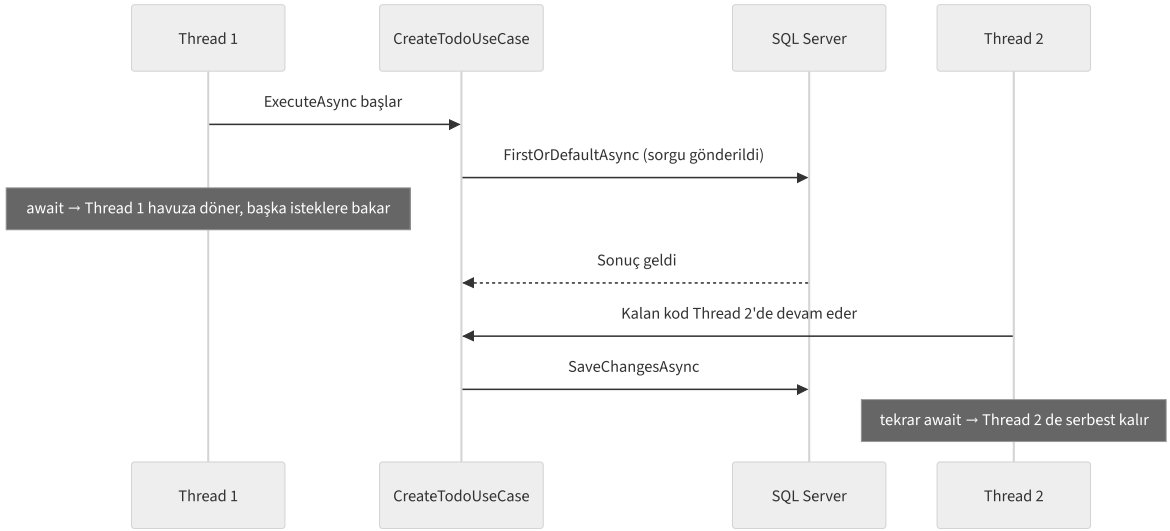
BENZETME: GARSON

Bir restoranda garson siparişi mutfağa iletir. Senkron çalışan bir garson yemek pişene kadar mutfak kapısında bekler. O sırada başka masalara bakamaz, restoran kalabalıklaşınca yeni garson almak gerekir. Asenkron çalışan garson siparişi bırakır, başka masalara bakar, yemek hazır olunca gelip alır. Garson thread'tir, mutfak ise veritabanı, mail sunucusu ya da Google'dır.

I/O beklemesi ve thread pool

Bir API isteğinin süresinin büyük kısmı CPU'da değil, **beklemede** geçer: veritabanından cevap beklemek, SMTP sunucusunu beklemek, Google'ın açık anahtarlarını beklemek. Buna **I/O-bound** iş denir. ASP.NET Core istekleri **thread pool**'daki sınırlı sayıda thread ile işler. Bir thread veritabanını beklerken bloklanırsa, o süre boyunca başka hiçbir isteğe bakamaz.

`await`, bekleme sırasında thread'i havuza geri verir. Sonuç gelince işin kalanı havuzdaki herhangi bir boş thread'de devam eder. Aynı sayıda thread ile çok daha fazla eşzamanlı istek karşılanabilir.



Tek bir istek boyunca kod farklı thread'lerde çalışabilir. Bu normaldir ve ASP.NET Core'da güvenlidir.

await tam olarak ne yapıyor?

Derleyici her `async` metodu bir **durum makinesine** (state machine) çevirir. Bizim repository metodunu adım adım izleyelim:

1. `GetByIdWithTodosAsync` çağrılır, sorgu hazırlanır.

2. `await todoListQuery.FirstOrDefaultAsync(...)` satırına gelinir. EF Core sorguyu SQL Server'a gönderir ve henüz tamamlanmamış bir `Task` döndürür.
3. Task tamamlanmadığı için metot burada "durur". Kaldığı yer kaydedilir ve çağırana tamamlanmamış bir `Task<TodoList?>` döner. Use case de aynı şekilde durur, controller da. Zincir en üste kadar böyle gider.
4. Thread havuza döner.
5. SQL Server cevap verince kaldığı yerden devam edilir: `todoList` değişkeni dolar ve `return todoList;` çalışır.

Task, Task<T> ve ValueTask

- `Task` : değer döndürmeyen asenkron iş (`SaveChangesAsync` sarmalayıcıları, `SendAsync`).
- `Task<T>` : sonunda `T` döndüren iş (`Task<TodoItemDto?>`).
- `ValueTask<T>` : çoğu zaman beklemeden, senkron tamamlanan metotlar için daha hafif bir alternatif. ASP.NET'in `IExceptionHandler.TryHandleAsync` sözleşmesi bunu kullanır, çünkü handler'ların çoğu "bu benim hatam değil" deyip anında `false` döner.

"Async all the way" ve tuzakları

ASLA YAPMA

- `.Result` **veya** `.Wait()` **ile beklemek.** Thread'i bloklar, asenkronluğun bütün faydasını yok eder. Yük altında thread pool tükenir (thread starvation) ve uygulama donar. Bazı ortamlarda doğrudan deadlock olur. Zincirin her halkası `await` kullanmalı. Bu yüzden controller'dan repository'ye kadar her şey `async` .
- `async void` **metot yazmak.** Hata fırlatırsa yakalanamaz ve uygulamayı çökertebilir. Olay (event) işleyicileri dışında her zaman `async Task` yaz.
- **Aynı** `DbContext` **ile paralel sorgu atmak.** `Task.WhenAll(repo.A(), repo.B())` gibi bir kod "A second operation was started on this context instance" hatası verir. `DbContext` thread-safe değildir, sorgular sırayla `await` edilmeli.

Derinlemesine: ConfigureAwait(false) neden projede yok?

Eski ASP.NET'te ve masaüstü uygulamalarda bir "SynchronizationContext" vardı. `await` 'ten sonra kod özel bir thread'e (UI thread'i gibi) dönmeye çalışırdı, `ConfigureAwait(false)` da buna gerek olmadığını söylerdi. ASP.NET Core'da bu context yok, dolayısıyla uygulama kodunda `ConfigureAwait(false)` yazmanın bir etkisi yok. Başkalarının kullanacağı genel amaçlı kütüphaneler yazarken hâlâ önerilir.

Frontend'de asenkronluk: tek thread, event loop

Tarayıcıda JavaScript **tek bir thread**'te çalışır. Bekleme işini tarayıcı üstlenir. Cevap gelince yapılacak iş (callback) bir kuyruğa konur ve **event loop** sırası gelince onu çalıştırır. Angular'da iki asenkron yapı birlikte kullanılıyor:

	PROMİSE (ASYNC/AWAİT)	OBSERVABLE (RXJS)
Kaç değer	Tek bir değer	Sıfır, bir veya çok değer
Ne zaman başlar	Oluşturulduğu anda	Abone olunduğunda (tembel, "cold")
İptal	İptal edilemez	`unsubscribe` ile iptal edilir, HTTP isteği de kesilir
Projede	Component metotları: `async createTodo()`	`HttpClient`, interceptor, `AuthService`

İkisi arasındaki köprü `firstValueFrom(observable)`: Observable'a abone olur, ilk değeri bir Promise olarak verir ve aboneliği kapatır. Component'lerde okunaklı `async/await` yazmamızı sağlayan bu.

`ngOnInit` içindeki `void this.loadTodoLists();` satırında `void` bilinçli: "Bu Promise'i bekleme, arkada çalışsın" anlamına geliyor. `ngOnInit` beklenemeyen bir metot olduğu için böyle yazılır, hatalar da `loadTodoLists`'in kendi `try/catch`'inde yakalanır.

06 CancellationToken mekanizması

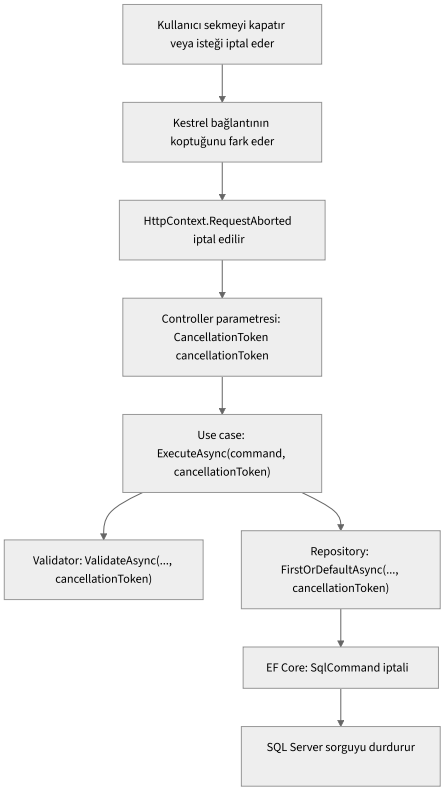
Hemen her asenkron metodun son parametresi `CancellationToken cancellationToken`. Bu parametre, artık sonucu kimsenin beklemediği bir işi **yarıda bırakabilmek** için var.

İki parça: kaynak ve token

- `CancellationTokenSource`: iptal etme yetkisi olan taraf. `Cancel()` metodu vardır.
- `CancellationToken`: sadece "iptal istendi mi?" diye **sorabilen** taraf. Elden ele dolaştırılan bu.

Bu ayrım bilinçli: token'ı alan bir metot işi kendisi iptal edemez, sadece iptal isteğine uyar. Bu yüzden mekanizmaya **işbirlikçi iptal** (cooperative cancellation) denir. Kimse bir işi zorla öldürmez. İş, token'ı kontrol ederek veya token'ı alt metotlara geçirerek kendisi durur ve `OperationCanceledException` fırlatır.

Projede token nereden geliyor, nereye gidiyor?



ASP.NET her istek için bir kaynak oluşturur. Action metodunda `CancellationToken` parametresi görünce oraya `HttpContext.RequestAborted`'ı otomatik koyar. Gerisi bizim token'ı aşağı doğru elden ele vermemize bağlı.

Token zincirin herhangi bir yerinde düşürülürse, örneğin repository'de `FirstOrDefaultAsync(predicate)` token'sız çağrılırsa, kullanıcı gitmiş olsa bile veritabanı sorgusu sonuna kadar çalışır ve kaynak harcar. Bu yüzden kuralımız: **token alan her metot onu, token alan her alt çağrıya geçirir.**

`= default` ne demek?

APPLİCATİON use case imzası

```
public async Task<TodoItemDto?> ExecuteAsync(
    CreateTodoCommand command,
    CancellationToken cancellationToken = default)
```

`default`, `CancellationToken.None` ile aynıdır: hiçbir zaman iptal edilmeyen bir token. Çağıran taraf token vermezse metot yine çalışır. Use case'lerde bu kolaylık var, repository interface'lerinde yok: altyapıya inen çağrıların token'ı unutmaması için orada zorunlu bırakıldı.

Projedeki diğer token kaynakları

YER	TOKEN'I KİM VERİYOR	NE ZAMAN İPTAL OLUR
Controller action'ları	ASP.NET (`RequestAborted`)	İstemci bağlantıyı kopardığında
`TodoReminderJob.RunAsync`	Hangfire	Sunucu kapanırken veya iş dashboard'dan silindiğinde. Kodda `CancellationToken.None` yazılı ama Hangfire çalışırken onu kendi token'ıyla değiştirir.
Exception handler'lar	ASP.NET	Hata yanıtı yazılırken istemci gittiğinde
Testler	xUnit: `TestContext.Current.CancellationToken`	Test zaman aşımına uğradığında veya çalıştırma durdurulduğunda

İptal ile gerçek hatayı ayırmak

Hatırlatma use case'i bir mail gönderilemezse diğerlerine devam eder. Ama iptal bir "mail hatası" değildir. Aşağıdaki `when` filtresi, iptal istisnasının hata listesine yazılmadan yukarı çıkmasını sağlıyor:

APPLİCATİON

todo.Application/Features/Reminders/SendDueTodoReminders/SendDueTodoRemindersUseCase.cs

```
foreach (TodoReminderCandidate candidate in candidates)
{
    cancellationToken.ThrowIfCancellationRequested();   // her görevden önce sor: durmam istendi mi?

    try
    {
        await _emailSender.SendAsync(message, cancellationToken);
    }
    catch (Exception exception) when (exception is not OperationCanceledException)
    {
        failures.Add(new ReminderFailure(candidate.Todo.Id, candidate.OwnerUserId, exception));
        continue;
    }

    candidate.Todo.MarkReminderSent(_timeProvider.GetUtcNow().UtcDateTime);
    await _todoReminderRepository.SaveChangesAsync(cancellationToken);
}
```

Derinlemesine: iptal her zaman iyi mi? "Yarım kalan iş" problemi

Yukarıdaki döngüde mail gönderildikten hemen sonra iptal gelirse `SaveChangesAsync` çalışmaz. Mail gitmiş ama "gönderildi" işareti kaydedilmemiş olur, sonraki turda aynı mail bir kez daha gidebilir. Buna **en az bir kez teslim** (at-least-once) denir. Hiç gitmemesindense iki kez gitmesini bilinçli olarak tercih ettik.

Bazı projeler "bu yazma işlemi yarıda kalmamalı" dediği kayıtlarda `CancellationToken.None` geçer. Böylece kullanıcı gitse bile kayıt tamamlanır. İptalin amacı boşa iş yapmamak. Tutarlılığı bozacak bir yerde iptali kabul etmemek de geçerli bir tasarım kararı.

Frontend'de iptal

Angular'da iptalin karşılığı **aboneliği bırakmak**. Bir `HttpClient` Observable'ından abonelik kaldırılırsa Angular tarayıcıdaki isteği de keser. Projedeki örnekler:

- `firstValueFrom` ilk değeri alınca aboneliği kendisi kapatır.
- `switchMap` yeni bir değer gelince önceki iç Observable'ı iptal eder. Interceptor, token yenilendikten sonra isteği yeni token'la göndermek için bunu kullanıyor.

07 Kimlik doğrulama (authentication)

Authentication tek bir soruyu cevaplar: **"Sen kimsin?"** Projede bu sorunun üç cevap yolu var: e-posta ve şifre, Google hesabı, ve daha önce verilmiş bir token. Sonraki bölümdeki authorization ise "Neye iznin var?" sorusunu cevaplar.

Şifreler nasıl saklanıyor?

Şifre **asla** düz metin olarak saklanmaz. Şifrelenmiş (encrypted) olarak da saklanmaz, çünkü şifrelenen şey anahtarla geri açılabilir. Bunun yerine tek yönlü bir **hash** saklanır:

- **PBKDF2-HMAC-SHA512, 210.000 iterasyon:** Bilerek yavaş bir algoritma. Doğru şifreyi bilen kullanıcı için tek hesaplama birkaç milisaniye sürer, milyarlarca tahmin deneyen bir saldırgan için ise bu süre katlanarak büyür. 210.000, OWASP'ın önerdiği sayı.
- **Salt:** Her şifreye rastgele bir ek değer katılır. İki kullanıcı aynı şifreyi seçse bile hash'leri farklı olur. Hazır hash tablolarıyla (rainbow table) kırılamaz. Testlerde aynı şifrenin iki kez hash'lenip farklı sonuç vermesi bu yüzden bekleniyor.
- **Hash yenileme:** Saklanan hash kullandığı iterasyon sayısını da içerir. Eski ayarla (100.000) üretilmiş bir hash'le başarılı giriş yapılırsa `SucceededRehashNeeded` döner ve hash sessizce güçlendirilir.

APPLİCATİON

todo.Application/Features/Authentication/LoginWithPassword/LoginWithPasswordUseCase.cs

```
if (user == null || user.PasswordHash == null)
{
    // Kullanıcı yokken de aynı maliyette bir hash hesaplanır; yanıt süresinden
    // "bu e-posta kayıtlı değil" bilgisi çıkarılamasın.
    _passwordHasher.HashPassword(providedPassword);

    throw new AuthenticationFailedException(InvalidCredentialsMessage);
}
```

"E-posta yanlış" ile "şifre yanlış" durumlarında hem mesaj hem yanıt süresi aynı. Aksi halde saldırgan sistemde hangi e-postaların kayıtlı olduğunu öğrenebilir. Buna **user enumeration** denir.

JWT: içi görülebilir ama değiştirilemez bir kimlik kartı

Giriş başarılı olunca API bir **JSON Web Token** üretir. JWT nokta ile ayrılmış üç parçadan oluşur: `başlık.içerik.imza`. İlk iki parça sadece Base64 ile kodlanmıştır, yani **herkes okuyabilir**. Projedeki bir token'ın içi şöyle görünür:

INFRASTRUCTURE JwtAccessTokenGenerator'ın ürettiği içerik (çözülmüş hali)

```
{ "alg": "HS256", "typ": "JWT" }
{
  "sub": "7",                    // kullanıcı numarası: ICurrentUser.UserId buradan okunur
  "email": "ali@example.com",
  "name": "Ali",
  "role": "User",                // yetkilendirme kararları için
  "jti": "6ea472ce-…",           // token'ın benzersiz numarası
  "iss": "TodoApi", "aud": "TodoWeb",
  "iat": 1791297666, "nbf": 1791297666, "exp": 1791298566   // 15 dakika geçerli
}
```

Üçüncü parça **imza**: ilk iki parçanın, sadece sunucunun bildiği gizli anahtarla (User Secrets'taki `Jwt:SigningKey`) HMAC-SHA256 ile hesaplanmış özeti. Biri `"role": "User"` değerini `"Admin"` yaparsa imza artık tutmaz ve API token'ı reddeder. Testlerde "başka bir anahtarla imzalanmış token reddedilir" senaryosu tam olarak bunu doğruluyor.

TUZAK

JWT şifreli değildir. İçine şifre, kimlik numarası gibi gizli bilgi konmaz. Gizli anahtar sızarsa herkes istediği token'ı üretebilir. Bu yüzden anahtar kodda veya `appsettings.json`'da değil, User Secrets'ta duruyor.

API'nin her istekte kontrol ettikleri:

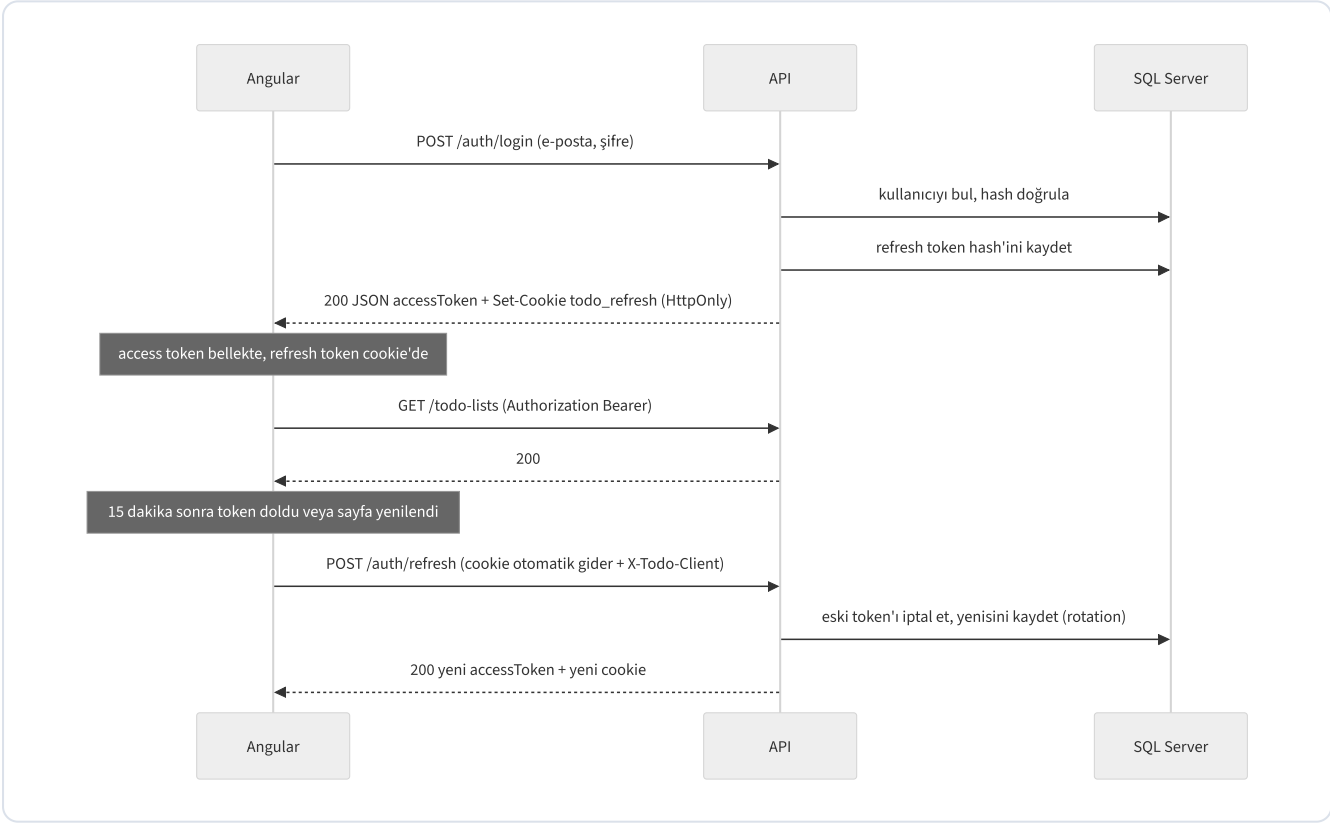
API todo.API/Authentication/AuthenticationSetup.cs

```
options.MapInboundClaims = false;   // claim adlarını "sub", "role" olarak bırak
options.TokenValidationParameters = new TokenValidationParameters
{
    ValidateIssuer = true, ValidIssuer = jwtSettings.Issuer,         // bizim API'miz mi üretmiş?
    ValidateAudience = true, ValidAudience = jwtSettings.Audience,   // bizim web uygulamamız için mi?
    ValidateIssuerSigningKey = true, IssuerSigningKey = new SymmetricSecurityKey(signingKeyBytes),
    ValidAlgorithms = new[] { SecurityAlgorithms.HmacSha256 },       // "alg: none" gibi saldırılara
kapalı
    ValidateLifetime = true, ClockSkew = TimeSpan.FromSeconds(30),  // süresi dolmuş mu?
    RoleClaimType = "role"
};
```

Neden iki token? Access token ve refresh token

	ACCESS TOKEN (JWT)	REFRESH TOKEN
Ömrü	15 dakika	7 gün
Ne işe yarar	Her API isteğinde kimliği kanıtlar	Sadece yeni bir access token almaya yarar
Tarayıcıda nerede	Angular servisinin belleğinde (değişkende)	HttpOnly cookie'de, JavaScript göremez
Sunucuda	Saklanmaz, imzasıyla doğrulanır	Veritabanında sadece SHA-256 hash'i saklanır
Çalınırsa	En fazla 15 dakika işe yarar	Rotation sayesinde ilk kullanımda fark edilir ve bütün oturumlar kapanır

Bu ikili yapı iki saldırı türü arasındaki dengeyi kurar. localStorage'daki bir token'ı sayfaya sızan zararlı bir script (**XSS**) okuyabilir. Cookie'deki bir değeri ise başka bir site kullanıcının tarayıcısı üzerinden gönderebilir (**CSRF**). Access token'ı bellekte tutup refresh token'ı korumalı bir cookie'ye koymak ikisine karşı da koruma sağlıyor.



Rotation ve çalıntı token tespiti

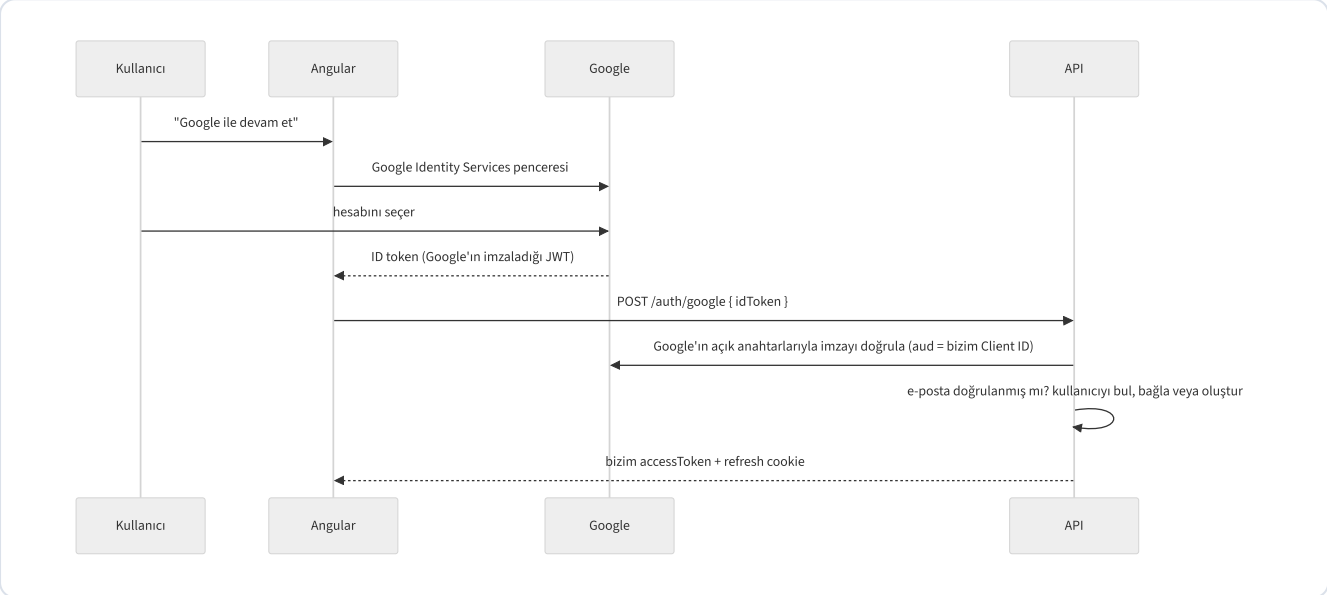
Her yenilemede eski refresh token iptal edilir ve yenisi verilir. İptal edilmiş bir token tekrar gelirse bunun tek açıklaması token'ın kopyalanmış olmasıdır: hem gerçek kullanıcı hem saldırgan aynı token'ı kullanmaya çalışıyordur. `RefreshSessionUseCase` bu durumda kullanıcının bütün oturumlarını kapatır. Tek istisna, aynı anda açılan iki sekmenin yarışı: az önce (30 saniye içinde) yenilenmiş bir token tekrar gelirse sadece o istek reddedilir.

Cookie'nin özellikleri

- `HttpOnly` : JavaScript `document.cookie` ile göremez.
- `Secure` : sadece HTTPS ile gider. Tarayıcılar `localhost` 'u güvenli saydığı için geliştirmede de çalışır.
- `SameSite=Strict` : başka bir siteden başlatılan isteklere eklenmez. CSRF'e karşı ilk savunma bu.
- `Path=/api/auth` : sadece giriş, yenileme ve çıkış isteklerinde gönderilir, görev istekleriyle hiç gitmez.

İkinci CSRF savunması `X-Todo-Client` başlığı. Başka bir site tarayıcıdan basit bir form isteği gönderebilir ama özel bir başlık ekleyemez. Özel başlıklı istekler için tarayıcı önce CORS'a sorar ve CORS sadece bizim Angular adresimize izin verir. `RequireClientHeaderAttribute` bu başlık olmayan istekleri 400 ile reddediyor.

Google ile giriş (OpenID Connect)



Önemli ayrıntılar:

- **Audience kontrolü:** Token'ın `aud` alanı bizim Client ID'miz olmalı. Başka bir site için alınmış bir Google token'ı bizim uygulamamıza giriş için kullanılamaz.
- **E-posta doğrulanmış olmalı:** Google'ın "bu e-postanın sahibi olduğunu doğruladım" demediği bir hesap reddedilir. Aksi halde biri, başkasının e-postasıyla açılmış hesabı ele geçirebilir.
- **Kullanıcı eşleştirme:** E-posta değişebilir ama Google'ın `sub` değeri değişmez. Eşleştirme önce `GoogleSubjectId` ile yapılır, bulunamazsa aynı e-postalı şifreli hesaba bağlanır.
- Google'ın token'ı sadece bir kez, kimliği kanıtlamak için kullanılır. Sonrasında uygulama kendi token'larıyla çalışır.

Frontend tarafı

- `provideAppInitializer` **:** Uygulama açılırken, router ilk sayfaya karar vermeden önce `restoreSession()` ile `/auth/refresh` çağrılır. Cookie geçerliyse kullanıcı sayfa yenilense bile oturumda kalır.

- **Interceptor:** Token'ı ekler. Süresi dolmuşsa önce yeniler. API 401 dönerse bir kez yenileyip isteği tekrarlar. Yenileme de başarısız olursa oturumu bitirip giriş sayfasına yönlendirir.
- **Tek yenileme isteği:** Aynı anda beş istek token isterse beş ayrı `/refresh` gitmesin diye `refreshSession` içinde `shareReplay(1)` ile tek bir istek paylaşılıyor.

WEB todo.Web/src/app/services/auth.service.ts

```
refreshSession(): Observable<Session | null> {
  if (this.refreshInFlight === null) {
    this.refreshInFlight = this.http.post<AuthResult>(`${this.apiUrl}/auth/refresh`, null).pipe(
      map((result) => this.startSession(result)),
      catchError(() => { this.session.set(null); return of(null); }),
      finalize(() => { this.refreshInFlight = null; }),
      shareReplay(1),
    );
  }
  return this.refreshInFlight;
}
```

EK KORUMALAR

Giriş, kayıt ve Google uçlarında IP başına dakikada 10 deneme sınırı var (rate limiting, 429). Bu, deneme yanılma (brute force) saldırılarını yavaşlatır. Refresh ucunda sınır yok, çünkü 256 bitlik rastgele bir değer tahmin edilemez.

08 Yetkilendirme ve erişim seviyeleri

Kimliği doğrulanmış olmak her şeye erişebilmek demek değil. Authorization **"Neye iznin var?"** sorusunu cevaplar. Projede dört seviye var ve her biri farklı bir yerde uygulanıyor.

SEVİYE	KURAL	NEREDE UYGULANIYOR	REDDEDİLİRSE
0. Anonim	Herkes erişebilir	API `[AllowAnonymous]` : `/auth/config` , `/login` , `/register` , `/google` , `/refresh` , `/logout`	—
1. Giriş yapmış	Geçerli bir access token gerekir	API `FallbackPolicy` ve controller'lardaki `[Authorize]`	**401** Unauthorized
2. Kaynağın sahibi	Liste kullanıcıya ait olmalı (veya Admin)	APP `TodoListAccessPolicy` , her use case içinde	**404** Not Found
3. Rol	Admin rolü gerekir	API `[Authorize(Policy = AdminOnly)]`	**403** Forbidden

Kimlik token'dan koda nasıl ulaşıyor?

1. `UseAuthentication()` middleware'i token'ı doğrular ve içindeki claim'lerden bir `ClaimsPrincipal` oluşturur. Bu nesneye `HttpContext.User` ile erişilir.
2. `HttpContextCurrentUser` bu nesneden `sub` ve `role` değerlerini okur.
3. Use case sadece `ICurrentUser.UserId` ve `ICurrentUser.IsAdmin` 'i görür. HTTP'den, token'dan, claim'den habersizdir.

API `todo.API/Authentication/HttpContextCurrentUser.cs`

```
public int UserId
{
    get
    {
        ClaimsPrincipal principal = GetAuthenticatedPrincipal();
        string? subjectClaimValue = principal.FindFirstValue(JwtRegisteredClaimNames.Sub);
        // ...
        return userId;
    }
}

public bool IsAdmin
{
    get { return GetAuthenticatedPrincipal().IsInRole(UserRole.Admin.ToString()); }
}
```

Seviye 1: "Varsayılan olarak kapalı"

API todo.API/Authentication/AuthenticationSetup.cs

```
options.FallbackPolicy = new AuthorizationPolicyBuilder()
    .RequireAuthenticatedUser()
    .Build();
```

Fallback policy, hiçbir yetki attribute'u olmayan her endpoint'e uygulanır. Yarın biri yeni bir controller ekleyip `[Authorize]` yazmayı unutsa bile o endpoint açıkta kalmaz. Açık olması gereken endpoint'ler `[AllowAnonymous]` ile **bilerek** açılır. Güvenlikte buna "secure by default" denir: unutmanın sonucu güvensizlik değil, fazladan güvenlik olur.

Seviye 2: kaynak tabanlı yetkilendirme

APPLİCATİON todo.Application/Authorization/TodoListAccessPolicy.cs

```
public static bool CanAccess(TodoList todoList, ICurrentUser currentUser)
{
    if (currentUser.IsAdmin)
    {
        return true;
    }

    return todoList.IsOwnedBy(currentUser.UserId);
}
```

Bu kontrol neden API'deki bir attribute'la yapılmıyor? Çünkü kararı verebilmek için önce listenin veritabanından yüklenmesi ve sahibine bakılması gerekiyor. Middleware'in bunu bilmesi mümkün değil. Kural use case'te durduğu için aynı use case nereden çağrılırsa çağrılsın kontrol atlanamaz.

NEDEN 403 DEĞİL DE 404?

Başkasının 15 numaralı listesine erişmeye çalışan birine 403 dönseydik, "15 numaralı bir liste var ama senin değil" demiş olurduk. Saldırgan numaraları tek tek deneyerek sistemde kaç liste olduğunu öğrenebilirdi. 404, liste hiç yokmuş gibi davranır. Admin sayfasında ise 403 dönüyoruz, çünkü orada gizlenecek bir kaynak yok, sadece yetki eksik.

Seviye 3: rol tabanlı yetkilendirme

API todo.API/Controllers/AdminUsersController.cs

```
[ApiController]
[Route("api/admin/users")]
[Authorize(Policy = AuthorizationPolicies.AdminOnly)]
public sealed class AdminUsersController : ControllerBase
```

Policy adı `AuthorizationPolicies.AdminOnly` sabitinde, kuralı da `AuthenticationSetup`'ta (`RequireRole("Admin")`) duruyor. Doğrudan `[Authorize(Roles = "Admin")]` yazmak yerine policy kullanmak, kural ileride değişirse (örneğin "Admin veya Manager") tek bir yerden güncellenmesini sağlar. Kimin Admin olacağı kodda değil, yapılandırmada (`Authorization:AdminEmails`) duruyor.

Kim neyi yapabilir?

İŞLEM	ANONİM	KULLANICI	ADMİN
Giriş, kayıt, Google ile giriş	Evet	Evet	Evet
Kendi listelerini görmek ve değiştirmek	401	Evet	Evet
Başkasının listesini id ile açmak	401	404	Evet
Kullanıcı listesini görmek	401	403	Evet

Frontend guard'ları sadece kullanıcı deneyimi içindir

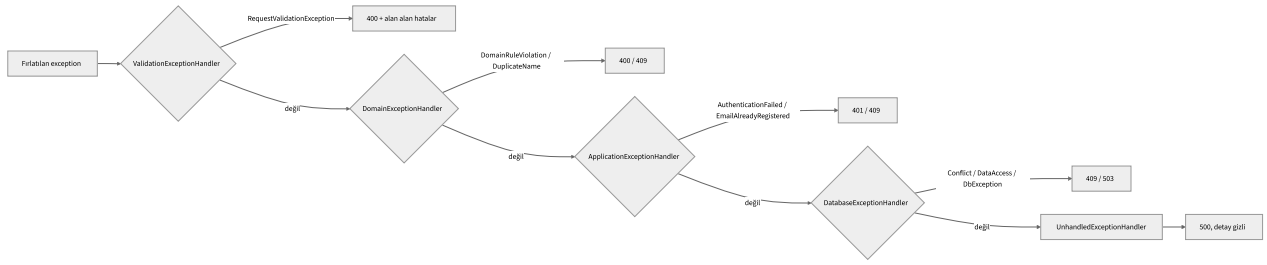
`authGuard`, `guestGuard` ve `adminGuard` kullanıcıyı doğru sayfaya yönlendirir. Ama tarayıcıdaki kodu kullanıcı değiştirebilir, geliştirici araçlarından guard'ı devre dışı bırakabilir. Gerçek güvenlik her zaman sunucuda: guard atlanıp admin sayfası açılsa bile `/api/admin/users` isteği 403 alır ve sayfa "Admin yetkisi gerekiyor" mesajı gösterir.

PATRON SORARSA: "YETKİLENDİRME NEREDE YAPILIYOR?"

"Üç yerde, üç farklı soru için. Giriş yapmış mı, rolü uygun mu: API'de, middleware ve policy ile. Bu kayıt onun mu: Application'da, use case içinde, çünkü kayda bakmak gerekiyor. Frontend guard'ları ise sadece yönlendirme yapıyor, güvenlik sağlamıyor."

09 Validation, hata yönetimi ve loglama

Patronunun ayrımı (mekanik, business ve veritabanı hataları) projede birebir karşılık buluyor. Kural basit: **her hata türü ortaya çıktığı katmanda tanımlanır, API'deki kendi handler'ı tarafından HTTP yanıtına çevrilir.**



Handler'lar `Program.cs`'teki kayıt sırasıyla denenir. Hatayı ilk tanıyan yanıtı yazar ve `true` döner.

HATA TÜRÜ	NEREDE DOĞAR	ÖRNEK	LOG SEVİYESİ
Validation	APP FluentValidation	Görev adı boş	Information
Business	DOMAİN ve APP	Aynı isimli görev, yanlış şifre	Information / Warning
Veritabanı	INFRA SQL hatası çevrilir	Unique index çakışması, bağlantı kopması	Warning / Error
Mekanik (beklenmeyen)	Her yerde	Null referans, bug	Error

Aynı kural neden üç yerde?

"Aynı listede aynı isimli görev olamaz" kuralı katman katman korunuyor:

1. **Validation:** Başlık boş mu, 120 karakteri aşıyor mu? Veritabanına gitmeden reddedilir.
2. **Domain:** `TodoList` kardeş görevlerin adlarına bakar. Kullanıcıya anlaşılır bir 409 verilir.
3. **Veritabanı:** Aynı anda gelen iki istek domain kontrolünü birlikte geçerse, `(TodoListId, Title)` unique index'i ikincisini durdurur. Gerçek ortamda 10 eşzamanlı istekle denediğimizde 8 isteği bu katman yakaladı.

Karşılaştırma büyük/küçük harfe duyarsız ve Türkçe kurallara göre yapılıyor ("İş" = "iş"). Domain'deki `NameComparison` bilerek veritabanının collation'ıyla (`Turkish_CI_AS`) aynı kuralı kullanıyor, böylece iki katman farklı karar vermiyor.

ProblemDetails ve traceId

Bütün hata yanıtları aynı standart biçimde (RFC 9457) döner:

API 400 yanıtı

```
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "detail": "'Liste adı' boş olmamalı.",
  "instance": "/api/todo-lists",
  "errors": { "Name": ["'Liste adı' boş olmamalı."] },
  "traceId": "4c1611bd522b20059adbbe651b6e82f5"
}
```

`traceId`, Serilog'un her log satırına yazdığı numarayla aynı. Kullanıcı bir hata bildirdiğinde `todo.API/logs/todo-YYYYMMDD.log` dosyasında bu numarayla arama yapılır ve o isteğin bütün kayıtları bulunur.

LOGLAMADA ASLA

Şifre, token, cookie veya istek gövdesi loglanmaz. E-posta adresi gibi kişisel veriler yerine kullanıcı numarası yazılır. Log dosyaları çoğu zaman uygulamanın kendisinden daha az korunur.

10 Veri erişimi: EF Core

Entity Framework Core bir **ORM** (Object-Relational Mapper): C# nesneleri ile SQL tabloları arasında çeviri yapar. LINQ ile yazdığın sorguyu SQL'e, SQL'den gelen satırları nesneye çevirir.

Change tracking: "Update" metodu neden yok?

`UpdateTodoUseCase`'e bakarsan hiçbir yerde `repository.Update(todo)` çağrısı yok. Sadece nesne değiştiriliyor ve `SaveChangesAsync` çağrılıyor:

APPLİCATİON todo.Application/Features/Todos/UpdateTodo/UpdateTodoUseCase.cs

```
todoList.RenameTodo(todoToUpdate, command.Title);
todoToUpdate.ChangeDescription(command.Description);
todoToUpdate.SetDueAtUtc(command.DueAtUtc);
todoToUpdate.ChangePriority(command.Priority);

await _todoListRepository.SaveChangesAsync(cancellationToken);
```

EF Core bir nesneyi veritabanından yüklediğinde onun bir kopyasını (snapshot) saklar. `SaveChangesAsync` çağrılınca nesnenin şu anki haliyle kopyayı karşılaştırır ve sadece değişen kolonlar için `UPDATE` üretir. Buna **change tracking** denir. Bu yüzden aynı `DbContext` örneğinin istek boyunca yaşaması (scoped) önemli.

Sadece okunacak veriler için takibe gerek yok. `GetAllByOwnerAsync` bu yüzden `AsNoTracking()` kullanıyor: kopya tutulmaz, sorgu daha hızlı ve az bellek kullanır.

Private alanlar ve EF Core

Domain sınıflarındaki setter'lar private, görev listesi de `private readonly List<TodoItem> _todos`. EF Core bunlara nasıl yazıyor?

- Private setter'lara EF Core reflection ile erişebilir.
- `Todos` property'si sadece okunabilir bir görünüm döndürüyor. `TodoDbContext`'te `HasField("_todos")` ve `PropertyAccessMode.Field` ile EF Core'a "yüklerken doğrudan bu alana yaz" deniyor.
- EF Core constructor'ı parametre adlarına göre eşleştirerek çağırabilir (`User(email, displayName, role)` gibi).

Diğer eşleme detayları

- **UTC dönüştürücü:** SQL Server'ın `datetime2` tipi tarihin UTC mi yerel mi olduğunu saklamaz. `UtcDateTimeConverter`, okunan her tarihi UTC olarak işaretler. Bitiş tarihi kuralı UTC beklediği için bu gerekli.
- **Enum'lar:** `Priority` ve `Role` veritabanında sayı olarak (`int`) saklanır.
- **Hesaplanan property'ler** (`HasPassword`, `IsAdmin`) `Ignore` ile tablo dışında bırakılır.

- **Index'ler:** Unique index'ler (e-posta, liste adı, görev adı, refresh token hash'i) kuralları korur. `DueAtUtc` index'i hatırlatma sorgusunu hızlandırır.

Migration'lar

Model değiştiğinde (yeni tablo, kolon, index) `dotnet ef migrations add Ad` komutu, modelin önceki hali ile şimdiki halini karşılaştırıp bir C# dosyası üretir. `dotnet ef database update` de bu dosyaları sırayla veritabanına uygular. Hangilerinin uygulandığı `__EFMigrationsHistory` tablosunda tutulur. Projede şu ana kadar 5 migration var. Bunlardan biri (`AddUsersAndTodoListOwnership`), sahipsiz kalan eski deneme verilerini silen elle yazılmış bir SQL satırı içeriyor.

`TodoDbContextDesignTimeFactory` sayesinde migration araçları API'yi hiç başlatmaz, JWT anahtarı gibi sırlara ihtiyaç duymaz. Sadece bağlantı cümlesi yeterli.

Dayanıklılık: otomatik tekrar deneme

`EnableRetryOnFailure` ile geçici veritabanı hataları (bağlantı kopması, deadlock, timeout) olduğunda sorgu en fazla 3 kez tekrar denenir. Veritabanı gerçekten kapalıysa denemeler biter ve kullanıcı detay içermeyen bir 503 alır.

Derinlemesine: repository pattern EF Core üzerinde gereksiz değil mi?

EF Core'un `DbContext` 'i zaten bir repository ve unit of work gibi davranır. Bu yüzden bazı ekipler ayrıca repository yazmaz. Bu projede repository'ler var, çünkü Clean Architecture'da Application katmanı EF Core'u tanımamalı. Repository interface'leri "use case'in veriden ne istediğini" (örneğin `ExistsWithNameForOwnerAsync`) iş diliyle ifade ediyor ve testlerde basit fake'lerle değiştirilebiliyor. Bedeli biraz fazla kod.

11 Arka plan işleri: Hangfire

Normalde kod sadece bir HTTP isteği gelince çalışır. Hatırlatma mailleri ise kimse istek atmadan, düzenli aralıklarla çalışmalı. Bunu Hangfire üstleniyor:

- **Zamanlama:** `Reminders:Schedule` ayarındaki cron ifadesi (`*/5 * * * *` , yani 5 dakikada bir) ile `TodoReminderJob` tekrarlayan iş olarak kaydedilir.
- **Kalıcılık:** İşler ve geçmişleri SQL Server'daki `HangFire` şemasında saklanır. Uygulama yeniden başlasa da kaybolmaz.
- **`[DisableConcurrentExecution]`:** Önceki çalışma bitmeden yenisi başlamaz. Birden fazla sunucu olsa bile veritabanında kilit tutulur.
- **`[AutomaticRetry(Attempts = 3)]`:** İş hata verirse Hangfire kendisi tekrar dener.
- **Dashboard:** `/hangfire` adresinde, sadece geliştirme ortamında ve sadece kendi bilgisayarından.

Mimari açıdan önemli olan, işin kendisinin (hangi görevlere mail gidecek, mail nasıl yazılacak, ne zaman "gönderildi" işaretlenecek) Application'daki `SendDueTodoRemindersUseCase`'te olması. `TodoReminderJob` sadece "ne zaman" sorusuyla ilgilenip use case'i çağırıyor. Yarın Hangfire yerine başka bir zamanlayıcı kullanılsa iş mantığı değişmez.

Güvenilirlik kuralı: bir görev ancak mail **başarıyla gittikten sonra** işaretlenir. Gönderilemeyen mail işaretlenmez ve sonraki turda tekrar denenir. Bitiş tarihi değişirse işaret sıfırlanır.

12 Frontend temelleri: Angular

Uygulama nasıl başlıyor?

1. Tarayıcı `index.html`'i yükler. İçinde sadece `<app-root>` etiketi var.
2. `main.ts` içindeki `bootstrapApplication(App, appConfig)` çalışır.
3. `app.config.ts`'teki provider'lar kurulur: HttpClient ve interceptor, router, app initializer, Türkçe yerel ayar.
4. App initializer (`restoreSession`) biter. Router adrese bakar, guard'ları çalıştırır ve doğru sayfayı `<router-outlet />` içine yerleştirir.

WEB todo.Web/src/app/app.config.ts

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideHttpClient(withInterceptors([authInterceptor])),
    provideRouter(routes),
    provideAppInitializer(() => inject(AuthService).restoreSession()),
    provideNzI18n(tr_TR),
    { provide: LOCALE_ID, useValue: 'tr-TR' },
  ],
};
```

Angular'ın eski sürümlerindeki `NgModule`'ler yok. Her component **standalone**: ihtiyaç duyduğu diğer component'leri kendi `imports` listesinde belirtir.

Bir component'in anatomisi

Her sayfa üç dosyadan oluşuyor. `todo-page.ts` durumu ve davranışı tutar, `todo-page.html` şablondur, `todo-page.scss` sadece bu component'e uygulanan stillerdir (Angular stilleri component'e hapseder, başka sayfaya taşmaz). Şablonda kullanılan sözdizimi:

SÖZDİZİMİ	ANLAMI	PROJEDEN ÖRNEK
`{{ ifade }}`	Değeri metin olarak yazar. HTML'i otomatik kaçışlar (XSS'e karşı korur).	`{{ todo.title }}`
`[özellik]="ifade"`	Property binding: veriden görünüme	`[nzLoading]="isSavingTodo"`
`(olay)="metot()"`	Event binding: görünümden koda	`(click)="logout()"`
`[(ngModel)]="alan"`	İki yönlü bağlama: input ile alan birbirini günceller	`[(ngModel)]="newListName"`
`@if / @else`	Koşullu gösterim	`@if (selectedList === null)`
`@for (... ; track x.id)`	Liste döngüsü. `track`, hangi satırın hangi veriye ait olduğunu söyler, değişmeyen satırlar yeniden çizilmez.	`@for (todo of todos; track todo.id)`
`@switch / @case`	Çoklu koşul	Giriş sayfasındaki Google durumları

Change detection: ekran ne zaman güncellenir?

Bu proje **zoneless** çalışıyor, `package.json`'da `zone.js` yok. Eski Angular uygulamalarında zone.js her asenkron işi izler ve bitince ekranı kendiliğinden kontrol ederdi. Zoneless modda Angular'a "veri değişti" demek gerekir. Projede bunun iki yolu birlikte kullanılıyor:

- **Signal'ler (modern yol):** `signal()` ile oluşturulan değer okunduğu yerleri kendisi bilir ve değişince sadece onları günceller. Giriş sayfası, admin sayfası ve `AuthService` böyle yazıldı (`readonly mode = signal<LoginMode>('login')`, şablonda `mode()`). `computed()` türetilmiş değerler için (`isAdmin`), `effect()` ise yan etkiler için (Google butonunu çizmek) kullanılıyor.
- **Düz alanlar ve `markForCheck()`:** İlk yazılan `TodoPage` düz alanlar kullanıyor. Her asenkron işin sonunda `this.changeDetector.markForCheck()` çağırarak ekranı yenilettiriyor. Çalışıyor ama unutulursa ekran güncellenmez. Signal'lere geçmek bu riski kaldırır.

RxJS: projede kullanılan operatörler

OPERATÖR	NE YAPAR	PROJEDE NEREDE
`pipe(...)`	Operatörleri sırayla uygular	Her yerde
`tap`	Değeri değiştirmeden yan etki yapar	Giriş başarılı olunca oturumu başlatmak
`map`	Değeri dönüştürür	API yanıtını `Session`'a çevirmek
`switchMap`	Bir değerden yeni bir Observable'a geçer, öncekini iptal eder	Token yenilendikten sonra isteği göndermek
`catchError`	Hatayı yakalayıp başka bir değere veya hataya çevirir	Interceptor'da 401 yönetimi
`finalize`	Başarı veya hata, her durumda en sonda çalışır	Yenileme bitince `refreshInFlight`'ı temizlemek
`shareReplay(1)`	Tek isteği birden fazla aboneye paylaştırır	Eşzamanlı yenileme isteklerini teke indirmek

Interceptor zinciri

`HttpInterceptorFn` bir fonksiyondur: isteği ve "sonraki adım" (`next`) fonksiyonunu alır. İsteği değiştirmek isterse `request.clone(...)` ile yeni bir kopya oluşturur, çünkü `HttpRequest` nesneleri değiştirilemez (immutable). Sonra `next(yeniİstek)` ile yola devam eder ve dönen Observable'a operatör ekleyerek yanıtı da işleyebilir. Backend'deki middleware hattının frontend'deki karşılığı bu.

Router ve guard'lar

WEB `todo.Web/src/app/app.routes.ts (kısaltılmış)`

```
{ path: 'login', component: LoginPage, canActivate: [guestGuard] },
{ path: 'admin/users', component: AdminUsersPage, canActivate: [adminGuard] },
{ path: '', component: TodoPage, canActivate: [authGuard], pathMatch: 'full' },
{ path: '**', redirectTo: '' },
```

Guard `true` dönerse sayfa açılır. `router.createUrlTree(['/login'])` dönerse kullanıcı oraya yönlendirilir. `**` bilinmeyen her adresi yakalar.

Frontend güvenliği

- **Otomatik kaçışlama:** `{{ }}` ile yazılan her şey metin olarak gösterilir. Görev adına `<script>` yazılsa bile çalışmaz. `[innerHTML]` ile kullanıcı verisi basmak bu korumayı devre dışı bırakır, projede kullanılmıyor.
- **Content Security Policy:** `index.html`'deki politika sadece kendi dosyalarımızdan ve Google'ın giriş adresinden script yüklenmesine izin verir. Bir XSS açığı olsa bile saldırganın kodu başka bir sunucuya veri

gönderemez.

- **Token bellekte:** localStorage'da hiçbir şey yok. Uçtan uca test bunu doğruluyor.

Frontend testleri

- **Vitest ve TestBed:** Component ve servisler gerçek tarayıcı açılmadan, sahte bağımlılıklarla test edilir.
- **HttpTestingController:** Gerçek ağ isteği gitmez. Test "şu adrese şu istek gitmeli" diye kontrol eder ve yanıtı kendisi verir (`request.flush(...)`).
- **Playwright:** Gerçek Chromium, gerçek API ve geçici bir SQL veritabanıyla kullanıcının yaptığı her şeyi baştan sona dener.

13 Test stratejisi

Testler bir piramit gibi dizilir. Altta çok sayıda hızlı ve küçük test, üstte az sayıda yavaş ama gerçekçi test. Projedeki dağılım:

SEVİYE	SAYI	NE TEST EDER	GERÇEK VERİTABANI?
DOMAİN birim testleri	55	Entity kuralları: isim, e-posta, refresh token süresi, hatırlatma kuralı	Hayır
APPLİCATİON birim testleri	81	Use case akışları fake'lerle, validator'lar, yetki kuralları	Hayır
INFRASTRUCTURE testleri	31	EF eşlemesi ve index'ler, JWT üretimi, şifre hash'i, MailKit (sahte SMTP sunucusuyla)	Hayır
API testleri	41	Gerçek HTTP hattı: 401, 403, güvenlik başlıkları, cookie özellikleri, hata handler'ları	Hayır
ANGULAR testleri	25	Component, servis, interceptor, guard	Hayır
Uçtan uca (Playwright)	9	Kullanıcı senaryoları: kayıt, oturum yenileme, izolasyon, aynı isim, CSRF	Evet, her çalıştırmada yeni ve geçici

Bir testin adı ne doğruladığını söyler. Örneğin `Refresh_RevokesEverySessionWhenAnOldTokenIsReused` okunduğunda kural anlaşılır. Testler aynı zamanda çalışan bir dokümantasyon: bir kuralın tam olarak nasıl işlediğini merak ettiğinde ilk bakacağın yer testi.

TESTLER PROJEDE GERÇEKTEN NEYİ YAKALADI?

Refresh hatasında cookie'yi silen başlığın exception handler tarafından sıfırlandığı bir hatayı API testi yakaladı. Hangfire dashboard'unun CSP yüzünden bozuk göründüğü hatayı da tarayıcı kontrolü yakaladı. İkisi de elle deneyerek fark edilmesi zor hatalardı.

14 Kendini test et

Önce cevabı kendin düşün, sonra altındaki cevapla karşılaştır. Bu sorular patronunun sorabileceği türden.

Soru **1. Application katmanı neden Infrastructure'a referans vermiyor? Peki use case veritabanına nasıl ulaşıyor?**

Cevap: Bağımlılık kuralı: kod sadece içe doğru bağımlı olabilir. Application, ihtiyacını bir interface (`ITodoListRepository`) olarak tanımlar. Infrastructure bu interface'i implement eder. Çalışma anında DI container use case'e gerçek repository'yi verir. Kod bağımlılığı içe, çalışma akışı dışa doğrudur.

Soru **2. TodoItem.Rename neden internal?**

Cevap: "Aynı listede aynı isimli görev olamaz" kuralını ancak listenin kendisi uygulayabilir, çünkü tek bir görev kardeşlerini bilmez. Yeniden adlandırmanın tek yolu aggregate root olan `TodoList.RenameTodo` olsun diye `Rename` Domain projesi dışına kapatıldı.

Soru **3. TodoDbContext neden singleton değil de scoped?**

Cevap: DbContext bir isteğin değişikliklerini takip eder (change tracking) ve thread-safe değildir. Singleton olsaydı bütün kullanıcılar aynı nesneyi paylaşır, bir kullanıcının yarım kalmış değişikliği başkasının SaveChanges'ıyla kaydedilebilir ve eşzamanlı isteklerde hata alınırdı.

Soru **4. Bir exception handler'a ITodoListRepository enjekte edilebilir mi?**

Cevap: Hayır. Exception handler'lar singleton olarak kaydediliyor, repository ise scoped. Singleton'ın scoped bir şeye bağımlı olması "captive dependency" hatasıdır. ASP.NET Development ortamında uygulama bu yüzden başlamaz.

Soru **5. await bir thread'i bekletir mi?**

Cevap: Hayır. Beklenen iş henüz bitmediyse metot o noktada durur, tamamlanmamış bir Task döndürür ve thread havuza geri döner. Sonuç gelince işin kalanı havuzdaki herhangi bir thread'de devam eder. Thread'i bekleten `.Result` ve `.Wait()`'tir, bu yüzden kullanılmazlar.

Soru **6. Repository'de CancellationToken'ı EF Core'a geçirmeyi unutursak ne olur?**

Cevap: Kullanıcı sekmeyi kapatsa bile SQL sorgusu sonuna kadar çalışır, veritabanı ve sunucu boşa kaynak harcar. Zincirin bir halkası token'ı düşürürse ondan sonraki hiçbir şey iptal edilemez.

Soru **7. JWT'nin içindeki "role" değerini değiştirerek Admin olunabilir mi?**

Cevap: Hayır. İçerik okunabilir ama değiştirilirse imza tutmaz. İmza sadece sunucunun bildiği gizli anahtarla üretilebilir. API imzayı her istekte doğrular ve uymayan token'ı 401 ile reddeder.

Soru **8. Access token neden localStorage'da değil?**

Cevap: Sayfaya sızan zararlı bir script (XSS) localStorage'ı okuyup token'ı çalabilir. Access token bellekte ve 15 dakikalık tutuluyor. Uzun ömürlü refresh token ise JavaScript'in hiç göremediği bir HttpOnly cookie'de duruyor.

Soru **9. İptal edilmiş bir refresh token tekrar gelirse API ne yapar, neden?**

Cevap: Normal kullanımda bu olmaz. Token kopyalanmış, yani çalınmış olmalı. API kullanıcının bütün aktif oturumlarını kapatır (reuse detection). İstisna: son 30 saniye içinde yenilenmiş bir token tekrar gelirse aynı anda açılmış iki sekme olarak kabul edilir, sadece o istek reddedilir.

Soru **10. Başkasının listesine erişmeye çalışan kullanıcı neden 403 değil 404 alıyor?**

Cevap: 403, "bu kaynak var ama senin değil" demek olurdu ve saldırgan numaraları deneyerek hangi listelerin var olduğunu öğrenebilirdi. 404 kaynağın varlığını gizler.

Soru **11. Yeni bir controller'a [Authorize] yazmayı unutursak endpoint açıkta kalır mı?**

Cevap: Hayır. FallbackPolicy, yetki attribute'u olmayan her endpoint'e "giriş yapmış kullanıcı" şartını uygular. Açık olması gereken endpoint'ler [AllowAnonymous] ile bilerek açılır (secure by default).

Soru **12. Angular'daki adminGuard kaldırılırsa normal bir kullanıcı kullanıcı listesini görebilir mi?**

Cevap: Hayır. Sayfa açılır ama `/api/admin/users` isteği API'deki AdminOnly policy'si yüzünden 403 alır. Guard'lar sadece yönlendirme içindir, güvenlik sunucudadır.

Soru **13. "Aynı isimli görev" kuralı domain'de zaten varken veritabanındaki unique index neden gerekli?**

Cevap: Aynı anda gelen iki istek domain kontrolünü birlikte geçebilir, çünkü ikisi de kaydetmeden önce kontrol eder. Unique index ikincisini veritabanı seviyesinde durdurur. Infrastructure bu hatayı ConflictException'a çevirir, kullanıcı yine 409 alır.

Soru **14. UpdateTodoUseCase neden repository.Update(todo) çağırmıyor?**

Cevap: EF Core yüklediği nesneleri takip eder (change tracking). SaveChangesAsync, nesnenin şu anki halini yüklendiği andaki kopyasıyla karşılaştırır ve sadece değişen kolonları günceller.

Soru **15. TodoPage'de markForCheck() çağrısı neden gerekli, giriş sayfasında neden yok?**

Cevap: Proje zoneless: Angular asenkron işlerin bittiğini kendiliğinden bilmez. TodoPage düz alanlar kullandığı için değişiklikten sonra markForCheck() ile haber verir. Giriş sayfası signal kullanıyor. Signal değişince onu okuyan şablon kendiliğinden güncellenir.

Soru **16. Kullanıcı "500 hatası aldım" derse sorunu nasıl bulursun?**

Cevap: Hata yanıtındaki traceId'yi istersin (detail alanında da yazıyor). `todo.API/logs` altındaki günün log dosyasında bu numarayla arama yaparsın. O isteğin bütün kayıtları ve hatanın tam detayı (stack trace) orada.

Soru **17. Hatırlatma maili gönderildikten hemen sonra uygulama kapanırsa ne olur?**

Cevap: Mail gitmiş ama "gönderildi" işareti kaydedilmemiş olabilir. Sonraki turda aynı mail bir kez daha gider. Bu bilinçli bir tercih: hiç gitmemesindense iki kez gitmesi (at-least-once).

Soru **18. Validator ile domain kuralı arasındaki fark ne?**

Cevap: Validator gelen verinin biçimine bakar ve sadece veriye bakarak karar verebilir (boş mu, uzun mu, e-posta biçiminde mi). Domain kuralı mevcut durumu bilmeyi gerektirir (bu listede aynı isimli görev var mı) ve nesnenin hiçbir zaman geçersiz duruma düşmemesini garanti eder.

---

15 Sözlük

Aggregate root

Birlikte değişen nesneler grubunun dışarıya açılan tek kapısı. Projede `TodoList`.

Anemik model

Sadece get/set'ten oluşan, kuralları başka yerlerde duran entity. Zengin modelin tersi.

Authentication

Kimlik doğrulama: "Sen kimsin?"

Authorization

Yetkilendirme: "Neye iznin var?"

CancellationToken

Bir işin iptal edilmesinin istenip istenmediğini sorabilen yapı.

Change tracking

EF Core'un yüklediği nesnelerdeki değişiklikleri izleyip sadece onları kaydetmesi.

Claim

Token içindeki bir bilgi parçası (`sub`, `role`, `email`).

Command

Use case'e giren veriyi taşıyan sınıf.

Composition root

Bütün bağımlılıkların birbirine bağlandığı tek yer. Projede `Program.cs`.

CORS

Tarayıcının farklı kökenden gelen isteklere izin verilip verilmediğini sunucuya sorduğu mekanizma.

CSRF

Başka bir sitenin kullanıcının tarayıcısı üzerinden onun adına istek göndermesi.

CSP

Sayfanın hangi kaynaklardan içerik yükleyebileceğini belirleyen tarayıcı politikası.

DI container

Nesneleri oluşturan, bağımlılıklarını veren ve yaşam sürelerini yöneten yapı.

DTO

Katmanlar veya sistemler arasında sadece veri taşıyan nesne.

Fake

Bir interface'i testler için basit ama gerçekten çalışan şekilde implement eden sınıf.

HttpOnly cookie

JavaScript'in okuyamadığı, sadece tarayıcının isteklerle gönderdiği cookie.

Idempotent

Birden fazla kez yapıldığında da sonucu değişmeyen işlem (çıkış yapmak gibi).

Interceptor

Angular'da her HTTP isteğinin arasına giren fonksiyon.

JWT

İmzalı, içi okunabilir kimlik token'ı: başlık.içerik.imza.

Middleware

ASP.NET'te her isteğin sırayla geçtiği adımlardan biri.

Migration

Veritabanı şemasındaki bir değişikliği uygulayan, sürümlenmiş C# dosyası.

Observable

Zamanla değer üreten, abone olununca başlayan ve iptal edilebilen RxJS yapısı.

ProblemDetails

HTTP hata yanıtları için standart JSON biçimi (RFC 9457).

Rate limiting

Belirli bir sürede kabul edilen istek sayısını sınırlamak.

Refresh token rotation

Her yenilemede eski refresh token'ı iptal edip yenisini vermek.

Salt

Hash'lenmeden önce şifreye eklenen, kullanıcıya özel rastgele değer.

Scoped

İstek başına bir örnek yaşayan DI kaydı.

Signal

Angular'da değiştiğinde onu okuyan yerleri kendiliğinden güncelleyen değer.

Thread pool

.NET'in işleri dağıttığı, önceden oluşturulmuş thread havuzu.

traceId

Bir isteğin log kayıtlarını ve hata yanıtını birbirine bağlayan numara.

Use case

Uygulamanın yaptığı tek bir iş (görev oluştur, giriş yap).

Zoneless

zone.js olmadan, signal ve açık bildirimlerle çalışan Angular change detection modu.